Between Lines of Code: Unraveling the Distinct Patterns of Machine and Human Programmers

YULING SHI, Shanghai Jiao Tong University, China
HONGYU ZHANG, Chongqing University, China
CHENGCHENG WAN, East China Normal University, China
XIAODONG GU, Shanghai Jiao Tong University, China

The rapid evolution of large language models (LLMs) has transformed automated code generation, but it has also made it increasingly difficult to distinguish between human-written and machine-generated code. This ambiguity creates practical challenges for software integrity, code ownership, and security. While zero-shot detection tools like DetectGPT work well for natural language, they struggle with source code due to its strict syntactic rules. This paper reproduction summarizes the work of Shi et al., who conducted an empirical analysis comparing the stylistic differences between human and AI programmers. Their findings indicate that machine-generated code is typically more concise, uses a narrower vocabulary, and is highly predictable—particularly regarding the placement of whitespace and newlines. Based on these insights, the original authors developed DetectCodeGPT, a zero-shot detection framework that perturbs code by inserting random spaces and newlines rather than altering structural tokens. The reproduced evaluation demonstrates that this stylized perturbation approach outperforms existing baselines in identifying machine-generated code across multiple LLMs, without requiring an external perturbation model.

1 Introduction

The rise of large language models (LLMs) such as Codex [1] and ChatGPT [11] has changed how software is written. Trained on enormous code corpora scraped from open-source repositories, these models can now produce code that is often syntactically correct and functionally reasonable, and they are increasingly used throughout the software development lifecycle [12].

This convenience comes with a cost: it is no longer obvious whether a given piece of code was written by a human or generated by a model. Shi et al. [13] point out that this ambiguity is not a merely academic concern. It complicates bug triage and code-ownership attribution, it can inflate assessments of a developer's productivity, and it allows latent vulnerabilities to go unnoticed because reviewers over-trust code that "looks" machine-produced and therefore assume it is robust. More broadly, when the provenance of code is unclear, the transparency of the entire development process is put at risk.

A natural place to look for a solution is the literature on detecting machine-generated *text*. Zero-shot, perturbation-based detectors such as DetectGPT [9] have become the state of the art for that problem: they exploit the fact that machine-generated text tends to sit at a sharper local maximum of the model's likelihood function, so that small perturbations of the text cause a larger drop in log-probability than similar perturbations do for human-written text. The trouble is that this idea was designed for natural language, which tolerates a great deal of lexical and syntactic variation. Source code does not: it must obey a rigid grammar in order to even parse, let alone execute. As the original authors argue, this mismatch means that natural-language detectors transfer poorly to code, and it is exactly why a systematic study of *what actually distinguishes* machine- and human-authored code was still missing before this work.

Authors' Contact Information: Yuling Shi, Shanghai Jiao Tong University, Shanghai, China, yuling.shi@sjtu.edu.cn; Hongyu Zhang, Chongqing University, Chongqing, China, hyzhang@cqu.edu.cn; Chengcheng Wan, East China Normal University, Shanghai, China, ccwan@sei.ecnu.edu.cn; Xiaodong Gu, Shanghai Jiao Tong University, Shanghai, China, xiaodong.gu@sjtu.edu.cn.

This study addresses that gap directly by conducting a controlled empirical comparison of human- and machine-authored Python functions along three axes – lexical diversity, conciseness, and naturalness – and show that machine-generated code is measurably more concise and more "natural" (i.e., more predictable under a language model) than human code, that it draws from a narrower vocabulary, and that it still follows familiar programming idioms. Crucially, they find that this gap is largest not in ordinary tokens but in *stylistic* tokens: whitespace and newlines. Building on that observation, they propose **DetectCodeGPT**, a perturbation-based detector that – instead of calling out to an external masked-language model to rewrite tokens, as DetectGPT does – perturbs code directly by inserting spaces and newlines at randomly chosen positions. This removes the dependency on a second, external model and makes the method considerably cheaper to run while, according to the paper, also making it more effective.

Across two datasets and six open code LLMs ranging from 1.3B to 7B parameters, DetectCodeGPT is reported to outperform the strongest prior baseline by an average of 7.6% in AUROC, and it degrades only mildly when the model used for detection differs from the model that actually generated the code – an important property for real-world deployment, where the source model is rarely known.

The main contributions of this paper are threefold: (1) a systematic empirical study of the stylistic differences between LLM-generated and human-written code; (2) a new detection method, DetectCodeGPT, that exploits those differences through a stylized, model-free perturbation strategy; and (3) an extensive evaluation of that method across models, datasets, and ablations.

The remainder of this summary is organized as follows. Section 2 covers the background on code LLMs and on perturbation-based text detection that motivates the paper. Section 3 describes the empirical study design and the DetectCodeGPT algorithm itself. Section 4 reports the empirical findings and the detection performance results. Section 5 discusses why the method works, its limitations, and closes with a short reflection.

2 Background and Related Work

2.1 Large Language Models for Code

Modern code LLMs are, almost without exception, decoder-only Transformers [15] trained on very large text corpora. Codex [1] and AlphaCode were among the first to show that this recipe, applied to millions of GitHub repositories, produces models capable of writing functionally correct code from a natural-language description. Follow-up work improved on this basic recipe along two main lines: new pretraining objectives such as fill-in-the-middle, which teach a model to complete code given both a prefix and a suffix, and instruction fine-tuning, which aligns a model's outputs with what a user actually asks for. More recent general-purpose models such as ChatGPT and LLaMA are trained on a mixture of natural language and code and perform well on code generation despite not being code-specialized. Code Llama [12], the model family used as the code generator in the reproduced study, is one such model, further specialized for code through continued pretraining on code-heavy data.

2.2 Perturbation-Based Detection of Machine-Generated Text

This work builds directly on DetectGPT [9], adapting its core idea for code. DetectGPT assumes access to the log-probability function $\log p_\theta(\cdot)$ of the (candidate) source model. Given a passage x, it draws a set of small perturbations $\tilde{x} \sim q(\cdot \mid x)$ – originally, by masking spans of x and having a separate model such as T5 fill them back in – and computes the discrepancy

$$d(x, p_\theta, q) \triangleq \log p_\theta(x) - \mathbb{E}_{\tilde{x} \sim q(\cdot|x)} \left[\log p_\theta(\tilde{x}) \right]. \tag{1}$$

The intuition is that machine-generated passages sit near a local maximum of $\log p_\theta$: because the model wrote them, they are close to what the model considers highly probable, so any small rewriting tends to *lower* their probability noticeably, producing a large, positive value of d. Human-written passages, in contrast, reflect a far more varied distribution of experience and phrasing, so they are less tightly bound to any single model's notion of what is likely, and d tends to stay small. A sufficiently large $d(x, p_\theta, q)$ is therefore taken as evidence of machine authorship.

This idea is elegant for natural language, but it leans on an assumption that does not hold for code: that a random local rewrite of x is still a coherent, "in-distribution" example of the same kind of content. Natural language tolerates this – a masked-and-refilled sentence is usually still grammatical. Code frequently does not: masking and refilling a token can turn a valid program into one that no longer parses, changing $\log p_\theta$ for reasons that have nothing to do with authorship. This is precisely the gap that motivates the empirical study in Section 3 and the subsequent perturbation strategy in Section 3.3.

2.3 Related Detection Methods

Work on detecting machine-generated *text* broadly falls into two families: zero-shot methods, which use likelihood or rank statistics directly (as DetectGPT and its NPR-based successor DetectLLM [14] do) and require no labeled data, and supervised methods, which fine-tune a classifier such as RoBERTa on generations collected from specific model families. Zero-shot methods generalize better to unseen source models but are typically more computationally expensive per example, since most of them call an auxiliary model to generate perturbations.

Work specifically on machine-generated *code* detection is comparatively sparse. GPTSniffer [10] is a supervised baseline that fine-tunes CodeBERT to classify a snippet's authorship; DetectCodeGPT outperforms this supervised baseline on average, which is notable since GPTSniffer has the advantage of having seen labeled training examples. A separate line of work, code watermarking [6], takes an entirely different approach: rather than detecting authorship after the fact, it embeds an imperceptible marker into generated code during training or decoding. Watermarking can be very reliable when applicable, but it requires cooperation from whoever controls the generating model, which makes it unsuitable for the more general, model-agnostic detection setting that DetectCodeGPT targets.

3 Method

The methodology consists of two main parts. First, an empirical study is used to characterize how human- and machine-authored code differ along three dimensions. Second, these findings are translated into a concrete detection algorithm called DetectCodeGPT.

3.1 Empirical Study Design

The authors organize their comparison around three properties commonly used to describe coding style: *lexical diversity*, *conciseness*, and *naturalness*.

Lexical diversity is assessed with four metrics. *Token frequency* simply counts how often each distinct token occurs. *Syntax element distribution* groups tokens into seven coarse categories using the `tree-sitter` parser, reproduced in Table 1, and compares the proportion of each category between the two sources. *Zipf's law*, which states that a token's frequency is roughly inversely proportional to its frequency rank, and *Heaps' law*, which describes how the size of a corpus's vocabulary grows with corpus length, are both checked for machine- and human-written code to see whether machine code follows the same statistical regularities as human code, and how quickly its vocabulary grows.

Table 1. Studied categories of Python code tokens, as classified by the `tree-sitter` parser.

Category	Example tree-sitter types
keyword	`def`, `return`, `else`, `if`, `for`, …
identifier	`identifier`, `type_identifier`
literal	`string_content`, `integer`, `true`, …
operator	<, >, =, ==, +, …
syntactic symbol	`: ) ] [ ( , " { } .`
comment	`comment`
whitespace	space, newline

Conciseness is measured simply by the number of tokens and the number of lines in a snippet, which together capture both raw verbosity and how a developer chooses to lay code out across lines.

Naturalness, following the "naturalness of software" hypothesis [2] that programs are, statistically, at least as regular and predictable as natural language, is quantified with two per-token statistics under a language model p_θ: the *log likelihood* $\log p_\theta(x)$ of a token, and its *log rank* $\log r_\theta(x)$, i.e., the log-position of the token in the list of all vocabulary tokens sorted by the model's predicted probability at that position. Lower rank and higher likelihood both indicate a token the model considers highly predictable, i.e., more "natural."

3.2 Dataset and Experimental Setup for the Empirical Study

Human-authored code is drawn from 10,000 Python functions sampled from CodeSearchNet [4], a corpus of real-world functions scraped from open-source GitHub projects. For each function, its signature and docstring are used as a prompt to Code Llama-7B [12], and the model's completion is treated as the "machine-authored" counterpart to that same function's human-written body. Generation uses top-p (nucleus) sampling [3] with $p = 0.95$ at two temperatures, $T = 0.2$ and $T = 1.0$, capturing both a conservative and an exploratory generation regime, with completions capped at 512 tokens.

The choice of CodeSearchNet over curated benchmarks such as HumanEval is deliberate: HumanEval-style problems are short, self-contained programming exercises, whereas CodeSearchNet functions come from real production codebases with the messiness – and the individual style – that entails. This makes the comparison more representative of how machine- and human-written code actually differ "in the wild," at some cost to how well the model performs on any single function.

3.3 The DetectCodeGPT Algorithm

The empirical results (detailed in Section 4) show that the naturalness gap between machine- and human-written code is largest specifically in whitespace tokens. DetectCodeGPT turns this observation into a perturbation strategy: rather than perturbing arbitrary tokens with an external masked-language model, it perturbs only whitespace, directly and cheaply.

Problem formulation. Detection is framed as classifying whether a snippet x was produced by a source model p_θ. As in DetectGPT, this is done by perturbing x into $\tilde{x} \sim q(\cdot \mid x)$ and looking for a sharp change in a naturalness score.

Naturalness score. Instead of the raw log-likelihood gap of Equation 1, DetectCodeGPT adopts the Normalized Perturbed Log Rank (NPR) score, which prior work on text detection found to be smoother and more robust than a

likelihood-based score:

$$\mathrm{NPR}(x, p_\theta, q) \triangleq \frac{\mathbb{E}_{\tilde{x}\sim q(\cdot|x)}\left[\log r_\theta(\tilde{x})\right]}{\log r_\theta(x)}. \tag{2}$$

Intuitively, if perturbing x causes its average token rank to worsen by a lot relative to its own original rank, x is more likely to be machine-authored.

Perturbation strategy. Two lightweight, purely syntactic perturbations are used, chosen independently at random for each of k perturbed samples:

- *Space insertion.* Let C be the set of all candidate positions where a space could be inserted in x. A random subset $C_s \subseteq C$ of size $\alpha|C|$ is chosen, and at each chosen position c, a number of spaces

$$n_{\text{spaces}}(c) \sim \mathrm{Poisson}(\lambda_{\text{spaces}}) \tag{3}$$

  is inserted.
- *Newline insertion.* Symmetrically, x is split into lines L, a random subset $L_n \subseteq L$ of size $\beta|L|$ is chosen, and after each chosen line l, a number of newlines

$$n_{\text{newlines}}(l) \sim \mathrm{Poisson}(\lambda_{\text{newlines}}) \tag{4}$$

  is inserted.

Sampling the insertion counts from a Poisson distribution – rather than always inserting a fixed amount – is meant to imitate the irregular, unplanned way a human tends to space out their code, which is exactly the kind of variation Finding 5 in Section 4 shows is missing from machine-generated whitespace.

Because both perturbations only ever *add* whitespace, they cannot change the tokenization of the surrounding code or break its syntax – unlike DetectGPT's mask-and-refill perturbation, which can silently turn valid code into invalid code and corrupt the very signal the detector relies on.

Decision rule. Algorithm 1 summarizes the full procedure. The final score is the NPR of the original snippet minus the average NPR of its k perturbed variants; a score above a threshold ϵ is classified as machine-authored.

The threshold ϵ trades off false positives against false negatives and can be tuned per deployment; all evaluation in Section 4 instead reports threshold-independent AUROC.

4 Findings and Results

This section details two primary sets of results: the empirical patterns that distinguish human- from machine-authored code, and the detection performance of the DetectCodeGPT algorithm.

4.1 Patterns That Distinguish Human and Machine Code

Token frequency. Looking at the most frequent tokens in each corpus, both sources share the expected backbone of any Python program – punctuation like `.`, `(`, `)`, and control-flow keywords like `if`, `return`, `else` – which is unsurprising since the model itself was trained on human code and inherits its basic syntax. The more interesting gap is in exception-handling and object-oriented vocabulary: tokens such as `raise`, `TypeError`, and `isinstance` are noticeably more frequent in machine output, as are boilerplate identifiers like `__class__` and `__name__`. This suggests machine-generated code leans more heavily, and more explicitly, on defensive programming and object-oriented conventions than the human baseline does.

Algorithm 1: DetectCodeGPT: machine-generated code detection via stylized code perturbation

Input: code x; source model M; number of perturbations k; decision threshold ϵ; hyperparameters $\alpha, \beta, \lambda_{\text{spaces}}, \lambda_{\text{newlines}}$

Output: **true** if x is probably machine-authored, **false** otherwise

for $i \leftarrow 1$ **to** k **do**
  draw $p \sim \text{Uniform}(0, 1)$;
  if $p \leq 0.5$ **then**
    `// space insertion`
    let C = all candidate positions to insert a space in x;
    sample $C_s \subseteq C$ with $|C_s| = \alpha|C|$;
    foreach *position* $c \in C_s$ **do**
      $n_{\text{spaces}}(c) \sim \text{Poisson}(\lambda_{\text{spaces}})$;
      insert $n_{\text{spaces}}(c)$ spaces at c;
  else
    `// newline insertion`
    split x into lines L;
    sample $L_n \subseteq L$ with $|L_n| = \beta|L|$;
    foreach *line* $l \in L_n$ **do**
      $n_{\text{newlines}}(l) \sim \text{Poisson}(\lambda_{\text{newlines}})$;
      insert $n_{\text{newlines}}(l)$ newlines after l;
  store the perturbed code as $\tilde{x}_i$;
$\text{NPR}_x \leftarrow \text{NPR}(x, p_\theta, q) - \frac{1}{k}\sum_{i=1}^{k} \text{NPR}(\tilde{x}_i, p_\theta, q)$;
if $\text{NPR}_x > \epsilon$ **then**
  return ***true*** ; `// probably machine-authored`
else
  return ***false*** ; `// probably human-authored`

Finding 1. Machine-authored code shows more attention to exception handling and object-oriented idiom than human code, consistent with an emphasis on error prevention and adherence to common programming paradigms.

Syntax element distribution. Figure 1 breaks tokens down into the seven categories of Table 1 and compares their proportions between sources at both temperatures. Keyword, operator, symbol, and whitespace proportions are nearly identical between human and machine code, which the authors read as evidence that the foundational grammar of the language is preserved equally well by both. The categories that do differ, at a significance level of $p < 0.01$ under a chi-square test, are more telling: machine code uses a significantly *smaller* share of identifiers (suggesting more compact, less descriptively-named code), a slightly *larger* share of literals (suggesting more direct, hard-coded data manipulation), and, at the higher temperature, noticeably more comments – which the authors interpret as the model compensating for its own increased unpredictability at $T = 1.0$ with more explanatory text.

Finding 2. Machine-authored code tends to use fewer identifiers and more literals for direct data processing, and adds more comments as the generation temperature grows.

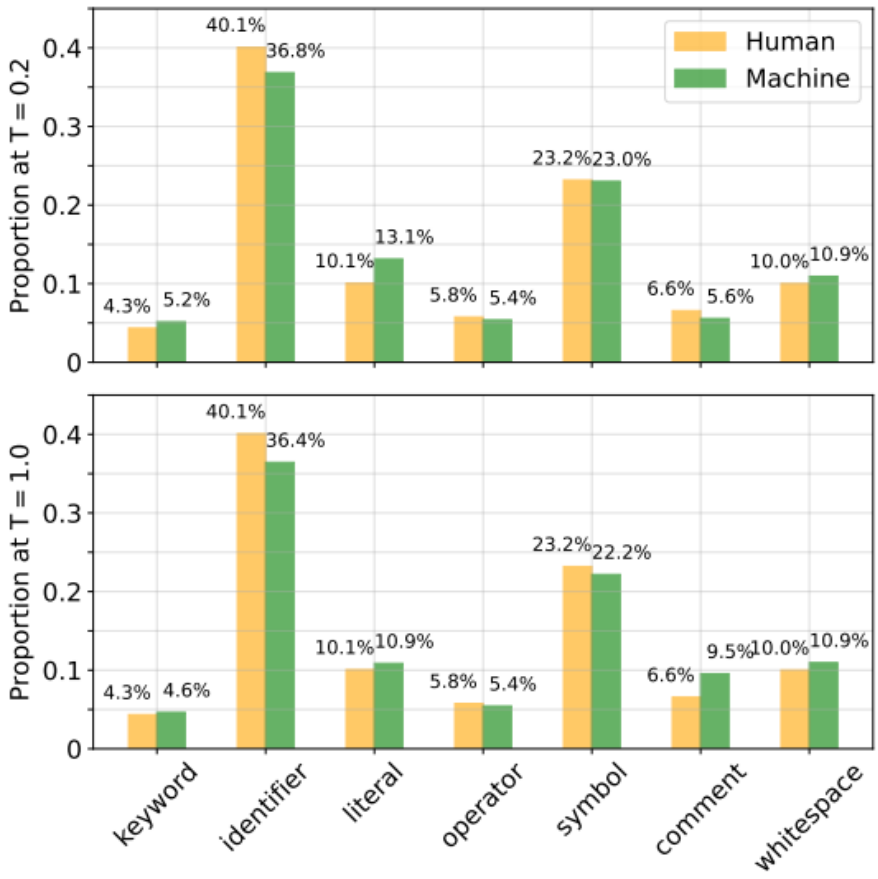

Fig. 1. Syntax element distribution of the code corpus, by token category and temperature.

Zipf's and Heaps' laws. Both corpora broadly obey Zipf's law – a small number of tokens dominate usage, with a long tail of rare ones – and Heaps' law – vocabulary size grows roughly as a power of corpus length. However, machine-generated code at the low temperature shows a visibly *shallower* Heaps' law slope, meaning its vocabulary grows more slowly as more code is generated, and a stronger concentration of probability mass on tokens ranked between roughly 10 and 100 under Zipf's law. The authors offer three complementary explanations: human variability and creativity naturally spreads token usage more broadly; a model sampled at low temperature is comparatively risk-averse, falling back on familiar, "safe" patterns; and some token sequences may simply be over-represented in the training data, biasing the model toward them.

> *Finding 3.* Machines prefer a narrower spectrum of frequently used tokens, while human code shows richer diversity in token selection.

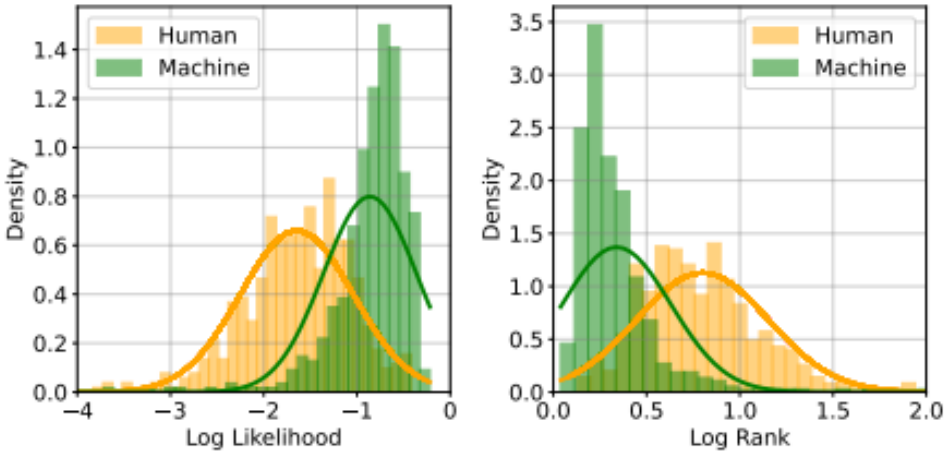

Fig. 2. Distribution of naturalness scores (log likelihood, left; log rank, right) for human- vs. machine-authored code at $T = 0.2$.

Conciseness. At $T = 0.2$, machine-generated functions are shorter than their human-written counterparts in both token and line count. This gap narrows considerably at $T = 1.0$, though machine code remains somewhat more concise on average. A plausible reading is that concise solutions were more common, or implicitly rewarded, during pretraining, whereas a human developer typically balances several competing goals – readability, anticipated future edits, comments for other developers – that tend to make code longer.

> *Finding 4.* Machine-authored code is more concise, especially at low temperature, while human-authored code tends to be longer, reflecting individual stylistic habits.

Naturalness. This is the result the whole detection method hinges on. Figure 2 shows that machine-generated code has systematically higher log-likelihood and lower log-rank than human code – i.e., it is more "natural" under the model, mirroring what has previously been observed for natural-language text. Table 2 breaks this naturalness gap down by token category. Every category shows a statistically significant gap ($p < 0.001$), but *whitespace* stands out with the single largest gap of any category in both log-likelihood and log-rank – larger even than the aggregate gap over all tokens combined.

This is the empirical justification for DetectCodeGPT's whole design: if whitespace usage is where machine and human code differ *most*, then a perturbation strategy that specifically targets whitespace should be a particularly sensitive probe of authorship.

> *Finding 5.* Machine-authored code is more "natural" than human-authored code overall, and this gap is most pronounced in tokens – comments and whitespace – that most directly reflect an individual's personal coding style.

Table 2. Naturalness of different syntax-element categories at $T = 0.2$; all differences are significant at $p < 0.001$.

	Log Likelihood			Log Rank		
Category	Mach.	Hum.	Δ	Mach.	Hum.	Δ
keyword	−1.701	−2.128	0.428	0.837	1.053	0.217
identifier	−0.459	−0.874	0.415	0.163	0.378	0.215
literal	−0.506	−1.364	0.858	0.152	0.630	0.479
operator	−0.938	−1.835	0.897	0.367	0.872	0.504
symbol	−0.868	−1.639	0.771	0.321	0.781	0.460
comment	−1.503	−3.028	1.525	0.608	1.610	1.002
whitespace	**−1.131**	**−2.740**	**1.609**	**0.429**	**1.441**	**1.012**
ALL	−0.827	−1.658	0.831	0.319	0.811	0.492

Table 3. Average AUROC across all 24 dataset/model/temperature combinations.

Method	**Avg. AUROC**
Entropy	0.3961
DetectGPT	0.6357
GPTSniffer (supervised)	0.6507
LRR	0.7050
NPR	0.7178
Rank	0.7228
Log $p(x)$	0.7649
Log Rank	0.7724
DetectCodeGPT (ours)	**0.8308**

4.2 Detection Performance

DetectCodeGPT is evaluated on held-out splits of CodeSearchNet and The Stack [5], against six code LLMs (Incoder, Phi-1, StarCoder [7], WizardCoder [8], CodeGen2, and Code Llama), at both temperatures, for 24 dataset/model/temperature combinations in total, using AUROC as the metric. Table 3 outlines the average AUROC across all 24 dataset/model/temperature combinations for every baseline. DetectCodeGPT achieves the highest average AUROC, showing a 7.6% average relative improvement over the strongest baseline, Log Rank, with the gap confirmed statistically significant by a Wilcoxon rank-sum test ($p < 0.001$) over 10 repeated runs. DetectCodeGPT is reported as the single best method in 20 of the 24 combinations. Notably, it also beats GPTSniffer, the *supervised* baseline, on average, despite using no labeled training data at all.

To see how this plays out per model rather than only on average, Table 4 highlights a slice of the full results table: performance on CodeSearchNet at $T = 0.2$, comparing DetectCodeGPT against the two strongest single-statistic baselines. The improvement is consistent across model sizes and families, and is largest for CodeGen2, where DetectCodeGPT gains more than 15 points of AUROC over Log Rank – the paper attributes CodeGen2's difficulty for other methods to it being trained on a larger share of natural-language text alongside code, which likely blurs its own notion of code "naturalness."

Across the board, detection is markedly harder at $T = 1.0$ than at $T = 0.2$: higher-temperature sampling makes machine-generated code more diverse and thus more human-like along exactly the dimensions the empirical study

Table 4. AUROC on CodeSearchNet ($T = 0.2$): representative baselines vs. DetectCodeGPT.

Code LLM	Log Rank	NPR	DetectCodeGPT
Incoder (1.3B)	0.9892	0.8143	**0.9896**
Phi-1 (1.3B)	0.7513	0.7566	**0.8287**
StarCoder (3B)	0.9340	0.9015	**0.9438**
WizardCoder (3B)	0.9120	0.8677	**0.9345**
CodeGen2 (3.7B)	0.7199	0.6177	**0.8802**
Code Llama (7B)	0.9016	0.5890	**0.9095**

Table 5. Ablation of perturbation strategies (Code Llama-7B, The Stack).

Temp.	MLM	Newline	Space	Newline&Space
$T = 0.2$	0.5436	0.8703	0.8639	**0.8852**
$T = 1.0$	0.6226	0.6453	0.6504	**0.6660**

Table 6. Effect of the number of perturbations k on AUROC.

k	10	20	50	100	200
$T = 0.2$	0.6537	0.8825	0.8852	**0.8855**	0.8846
$T = 1.0$	0.5558	0.6584	0.6660	0.6660	**0.6662**

measured, which shrinks the naturalness gap the detector relies on. DetectCodeGPT still leads at $T = 1.0$, but by a smaller margin.

4.3 Ablation and Sensitivity Analysis

Two ablations isolate why DetectCodeGPT works. Table 5 compares the two stylized perturbations, individually and combined, against the traditional masked-language-model (MLM) perturbation used by DetectGPT and DetectLLM. Both newline and space insertion alone already beat the MLM-based perturbation by a wide margin, and combining them performs best at both temperatures – direct evidence that the earlier empirical finding (whitespace carries the most authorship signal) translates into a better detector, and that avoiding an external rewriting model helps rather than hurts.

Table 6 sweeps the number of perturbations k used per snippet. AUROC rises sharply from $k = 10$ to $k = 20$ and then plateaus, with negligible gains beyond $k = 50$. This is a practically useful result: it means DetectCodeGPT does not need hundreds of perturbations (and therefore hundreds of extra forward passes) to work well, which keeps it cheap even without an external perturbation model.

4.4 Cross-Model Robustness

A realistic deployment scenario rarely has white-box access to the exact model that generated a suspect snippet. Figure 3 illustrates the cross-model experiment, where a *different* LLM is used as the surrogate scoring model p_θ than the one that actually generated the code. Performance drops only modestly outside the diagonal (white-box) entries: for instance, using StarCoder as the surrogate to detect code from WizardCoder or Code Llama achieves 0.92 and 0.87 AUROC

Fig. 3. Cross-model detection performance on The Stack at $T = 0.2$: rows are the source (generating) model, columns are the surrogate (detecting) model.

respectively, versus 0.93 for StarCoder detecting its own output. CodeGen2 is the hardest source model to detect in the black-box setting, again plausibly because of its more NL-heavy training mix, though Phi-1 does comparatively well as a surrogate for it (0.83 AUROC), hinting that ensembling several surrogate detectors could help further.

4.5 Qualitative Examples

Figure 4 walks through representative snippets and shows, qualitatively, what the quantitative results above mean in practice. In the machine-generated example, DetectCodeGPT correctly flags a snippet where related `if` statements are grouped together and separated from unrelated code by newlines – a modular, regularized layout its stylized perturbation is sensitive to. In two human-written examples, both baselines misclassify code that uses whitespace inconsistently (blank lines in some places but not others, or spaces added around some operators but not others), while DetectCodeGPT correctly identifies the irregularity as a human signature. The figure also includes a genuine failure case: a human-written snippet that happens to be unusually well-formatted is misclassified as machine-generated by every method, including DetectCodeGPT, underscoring that consistently tidy human code remains a hard case for this family of detectors.

5 Discussion

5.1 Why DetectCodeGPT Works

The paper attributes DetectCodeGPT's advantage over prior perturbation-based detectors to two factors, both of which follow naturally from the empirical study in Section 4. First, *correctness preservation*: because its perturbations only insert whitespace, they cannot silently break the syntax of the snippet the way masking-and-refilling a token can. An MLM-based perturbation occasionally substitutes the wrong identifier or introduces a type error, and that kind

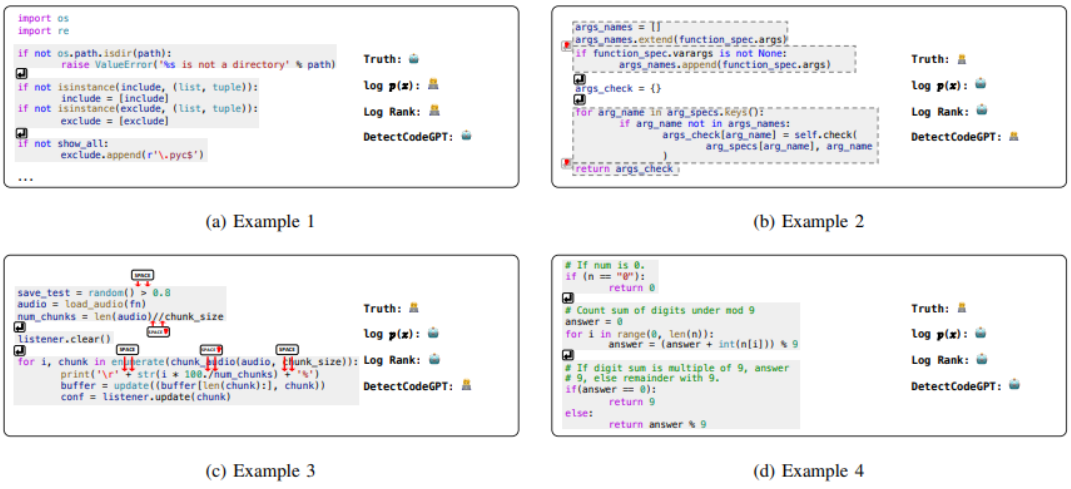

Fig. 4. Representative machine- and human-authored snippets with predictions from Log $p(x)$, Log Rank, and DetectCodeGPT.

of accidental corruption lowers the naturalness score for reasons that have nothing to do with whether the original snippet was human- or machine-written – adding noise exactly where the signal needs to be cleanest. Second, *fidelity to how humans actually vary their whitespace*: because Finding 5 shows whitespace is where the naturalness gap is largest, and because human whitespace usage is genuinely closer to random than a model's, sampling insertion counts from a Poisson distribution at randomly chosen locations mimics that human irregularity more faithfully than any single deterministic rule could.

5.2 Strengths

Relative to prior perturbation-based detectors, DetectCodeGPT does not need to query an external model to generate its perturbations, which removes both a source of latency and a second point of failure (what if that external model is unavailable, or itself biased in some way?). Relative to supervised detectors like GPTSniffer, it needs no labeled training data at all, and Section 4 shows it generalizes reasonably well even when the exact source model is unknown – a property a supervised classifier trained on one family of models cannot easily claim.

5.3 Limitations and Future Directions

The study explicitly mentions a few limitations. Computationally, their study is restricted to models of at most 7B parameters, so it remains unclear whether the same lexical, conciseness, and naturalness gaps – and therefore the same detector – hold up for today's much larger frontier models. Scope-wise, the entire empirical study and evaluation is conducted on Python; the authors argue, but do not verify, that the approach should transfer to languages such as C/C++, Java, and JavaScript, on the grounds that inserting whitespace is unlikely to change a program's meaning in those languages either. A further limitation visible in the results themselves is that detection accuracy at high temperature ($T = 1.0$) still leaves considerable room for improvement, since more exploratory sampling narrows exactly the naturalness gap the method depends on.

5.4 Reflection

The most convincing aspect of this methodology is arguably not just the final AUROC numbers, but the core diagnosis in Table 2 that motivates them: showing *specifically* that whitespace carries the largest naturalness gap of any token category turns DetectCodeGPT from an arbitrary design choice into a directly evidence-driven one. That said, the same diagnosis suggests a fairly obvious limitation the paper does not evaluate directly: if whitespace and formatting are the primary signal, a detector like this one is likely vulnerable to a user who simply runs a generated file through an autoformatter (e.g., `black` for Python) before sharing it, since that would erase exactly the irregularities the method is built to notice. Testing robustness to routine code formatting – something a developer might do for entirely benign reasons – seems like a natural next experiment, and a meaningful complement to the cross-model robustness study already in Section 4.

In summary, this study makes two primary contributions worth noting: a careful empirical account of how machine- and human-written code actually differ, and a lightweight detector, DetectCodeGPT, that turns the most discriminative part of that account – whitespace naturalness – into a practical, zero-shot, model-agnostic tool for judging code provenance.

References

[1] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. 2021. Evaluating Large Language Models Trained on Code. *arXiv preprint arXiv:2107.03374* (2021).

[2] Abram Hindle, Earl T. Barr, Mark Gabel, Zhendong Su, and Premkumar Devanbu. 2016. On the Naturalness of Software. *Commun. ACM* 59, 5 (2016), 122–131.

[3] Ari Holtzman, Jan Buys, Li Du, Maxwell Forbes, and Yejin Choi. 2019. The Curious Case of Neural Text Degeneration. In *International Conference on Learning Representations (ICLR)*.

[4] Hamel Husain, Ho-Hsiang Wu, Tiferet Gazit, Miltiadis Allamanis, and Marc Brockschmidt. 2020. CodeSearchNet Challenge: Evaluating the State of Semantic Code Search. In *arXiv preprint arXiv:1909.09436*.

[5] Denis Kocetkov, Raymond Li, Loubna Ben Allal, Jia Li, Chenghao Mou, Carlos Munoz Ferrandis, Yacine Jernite, Margaret Mitchell, Sean Hughes, Thomas Wolf, et al. 2022. The Stack: 3 TB of Permissively Licensed Source Code. *arXiv preprint arXiv:2211.15533* (2022).

[6] Taehyun Lee, Seokhee Hong, Jaewoo Ahn, Ilgee Hong, Hwaran Lee, Sangdoo Yun, Jamin Shin, and Gunhee Kim. 2023. Who Wrote This Code? Watermarking for Code Generation. In *arXiv preprint arXiv:2305.15060*.

[7] Raymond Li, Loubna Ben Allal, Yangtian Zi, Niklas Muennighoff, Denis Kocetkov, Chenghao Mou, Marc Marone, Christopher Akiki, Jia Li, Jenny Chim, et al. 2023. StarCoder: May the Source Be with You! *arXiv preprint arXiv:2305.06161* (2023).

[8] Ziyang Luo, Can Xu, Pu Zhao, Qingfeng Sun, Xiubo Geng, Wenxiang Hu, Chongyang Tao, Jing Ma, Qingwei Lin, and Daxin Jiang. 2023. WizardCoder: Empowering Code Large Language Models with Evol-Instruct. *arXiv preprint arXiv:2306.08568* (2023).

[9] Eric Mitchell, Yoonho Lee, Alexander Khazatsky, Christopher D. Manning, and Chelsea Finn. 2023. DetectGPT: Zero-Shot Machine-Generated Text Detection Using Probability Curvature. In *Proceedings of the 40th International Conference on Machine Learning (ICML)*, Vol. 202. 24950–24962.

[10] Phuong T. Nguyen, Juri Di Rocco, Claudio Di Sipio, Riccardo Rubei, Davide Di Ruscio, and Massimiliano Di Penta. 2023. Is This Snippet Written by ChatGPT? An Empirical Study with a CodeBERT-Based Classifier. *arXiv preprint arXiv:2307.09381* (2023).

[11] OpenAI. 2022. *ChatGPT: Optimizing Language Models for Dialogue.* Technical Report. OpenAI.

[12] Baptiste Roziere, Jonas Gehring, Fabian Gloeckle, Sten Sootla, Itai Gat, Xiaoqing Ellen Tan, Yossi Adi, Jingyu Liu, Tal Remez, Jérémy Rapin, et al. 2023. Code Llama: Open Foundation Models for Code. *arXiv preprint arXiv:2308.12950* (2023).

[13] Yuling Shi, Hongyu Zhang, Chengcheng Wan, and Xiaodong Gu. 2024. Between Lines of Code: Unraveling the Distinct Patterns of Machine and Human Programmers. *arXiv preprint arXiv:2401.06461* (2024).

[14] Jinyan Su, Terry Yue Zhuo, Di Wang, and Preslav Nakov. 2023. DetectLLM: Leveraging Log Rank Information for Zero-Shot Detection of Machine-Generated Text. *arXiv preprint arXiv:2306.05540* (2023).

[15] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. 2017. Attention Is All You Need. In *Advances in Neural Information Processing Systems (NeurIPS)*. 5998–6008.